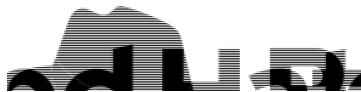


# Zap the Flakes!

Leveraging AI to Combat Flaky Tests with CANNIER

cloudland '25

[Daniel Hiller](#)



# agenda

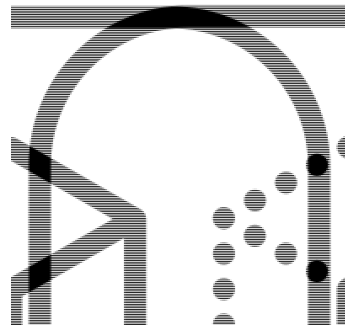
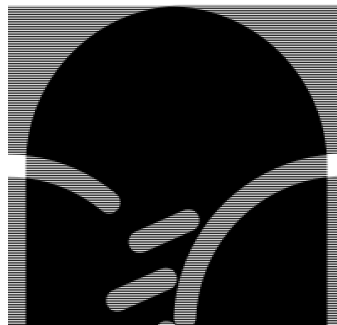
- about me
- about flakes
- pre-merge-detection v1
- CANNIER
- pre-merge-detection v2
- Q&A

# about me

- Software Engineer | [Red Hat OpenShift Virtualization](#)
- [KubeVirt](#) | CI & automation in general
- [prow.ci.kubevirt.io](#)
- [CI-Health](#)
- Flaky Tests



[kubevirt.io](#) - Virtualization for Kubernetes



about flakes

# about flakes

PR History: kubevirt/kubevirt #9445

|                                                   | 9d41878              |                     |                     |
|---------------------------------------------------|----------------------|---------------------|---------------------|
| pull-kubevirt-e2e-k8s-1.25-sig-compute-migrations | 1637934812398358528  | 1636633531918585856 | 1636403749595385856 |
| pull-kubevirt-e2e-k8s-1.25-sig-compute            | 1637934815221125120  | 1636403757321293824 |                     |
| pull-kubevirt-e2e-k8s-1.25-sig-network            | 1637934813975416832  | 1636403756985749504 |                     |
| pull-kubevirt-e2e-k8s-1.25-sig-operator           | 1637934815393091584  | 1636403757992382464 |                     |
| pull-kubevirt-e2e-k8s-1.25-sig-storage            | 1637934814088663040  | 1636633532048609280 | 1636403756704731136 |
| pull-kubevirt-e2e-k8s-1.26-sig-compute            | 1637934816471027712  | 1636404222087925760 |                     |
| pull-kubevirt-e2e-k8s-1.26-sig-network            | 16379348160853151744 | 1636403758015120844 |                     |

pull-kubevirt-e2e-k8s-1.25-sig-compute-migrations #1636633531918585856

Job History PR History Artifacts

Test started last Friday at 8:40 AM **passed** after 1h9m49s. ([more info](#))

JUnit

91/1406 Tests Passed!

1315/1406 Tests Skipped.

Build Log

Show all hidden lines [Raw build-log.txt](#)

pull-kubevirt-e2e-k8s-1.25-sig-compute-migrations #1636403749595385856

Job History PR History Artifacts

Test started last Thursday at 7:33 PM **failed** after 1h18m18s. ([more info](#))

JUnit

1/1406 Tests Failed.

Tests Suite: [rfe\_id:393][crit:high][vendor:cnv-qe@redhat.com][level:system][sig-compute] VM Live Migration [Serial]with a dedicated migration network Should migrate over that network 4m18s

90/1406 Tests Passed!

1315/1406 Tests Skipped.

source: <https://prow.ci.kubevirt.io/pr-history/?org=kubevirt&repo=kubevirt&pr=9445>

# about flakes

a ***flake***

is a **test** that

**without any code change**

will either **fail** or **pass** in successive runs

# about flakes

“... test flakiness was a **frequently encountered problem**, with

- **20%** of respondents claiming to experience it **monthly**,
- **24%** encountering it on a **weekly** basis and
- **15%** dealing with it **daily**”

source: [“A survey of flaky tests”](#)

# about flakes

“... In terms of severity, of the **91% of developers** who claimed to deal with flaky tests at least a few times a year,

- **56%** described them as a **moderate** problem and
- **23%** thought that they were a **serious** problem. ...”

source: [“A survey of flaky tests”](#)



# impact of flakes

from [“A survey of flaky tests”](#):

- **97%** of flakes were **false alarms\***, and
- **more than 50%** of flakes could not be **reproduced** in isolation

conclusion: “ignoring flaky tests is ok”

\*code under test actually is not broken, but it works as expected

## impact of flakes



# impact of flakes

# impact of flakes

in CI automated testing **MUST** give a **reliable signal of stability**

any failed test run signals that the product is **unstable**

test runs failed due to flakes do not give this reliable signal

**they only waste time**

# impact of flakes

Flaky tests waste everyone's time!

For the **project community** they cause

- **slowdown of merging** pull requests - “retest trap”
- **reversal of acceleration** effects (i.e. batch testing)
- **waste** of CI resources

# impact of flakes

Flaky tests waste everyone's time!

For **individual contributors** they cause

- **prolonged feedback cycles**
- **test trust issues**

pre-merge-detection v1

Goal:  
flaky tests must not  
enter the code base!



# pre-merge-detection v1

Initial approach:

- look at files touched in pull request
- determine changed files containing e2e tests
- run all contained tests **5 times** in **random** order

# pre-merge-detection v1

why:

- eliminate order-dependencies
- running five times is the boundary to find 88% of flaky tests



# pre-merge-detection v1

## [check-tests-for-flakes test lane](#)

why: catch flakes before entering  
main

([source](#))

```
ginkgo_params="$ginkgo_params -no-color -succinct --label-filter=!QUARANTINE -randomize-all"
for test_file in $(echo "${NEW_TESTS}" | tr '|' '\n'); do
    ginkgo_params+=" -focus-file=${test_file}"
done
|
echo "Test lane: ${TEST_LANE}, preparing cluster up"

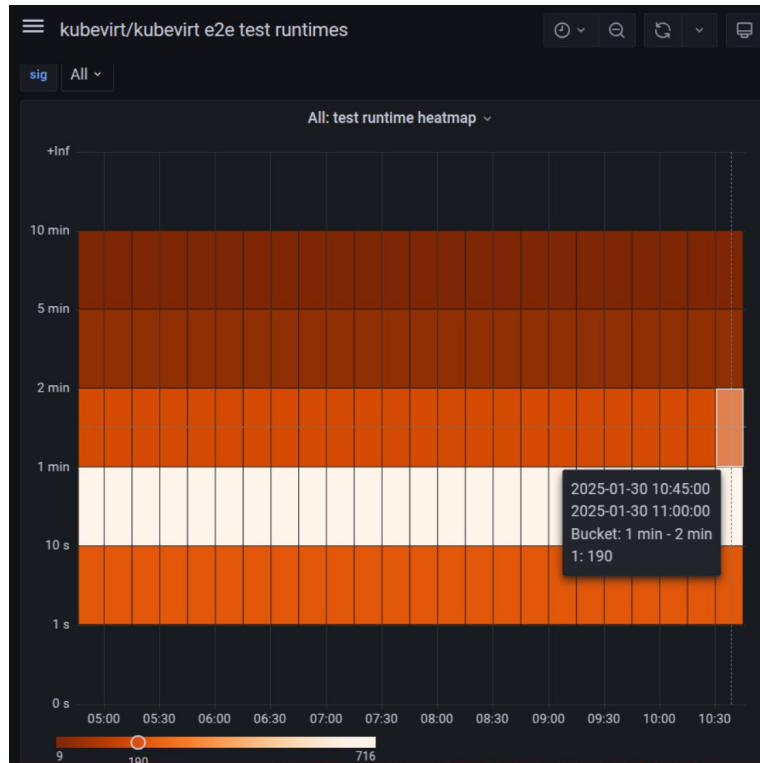
if [[ ! "$ginkgo_params" =~ -dry-run ]]; then
    make cluster-up
    make cluster-sync
else
    # Ginkgo only performs -dryRun in serial mode.
    export KUBEVIRT_E2E_PARALLEL="false"
    NUM_TESTS=1
fi

for i in $(seq 1 "$NUM_TESTS"); do
    echo "Test lane: ${TEST_LANE}, run: $i"
    if ! FUNC_TEST_ARGS="$ginkgo_params" make functest; then
        echo "Test lane: ${TEST_LANE}, run: $i, tests failed!"
        exit 1
    fi
done
```

# pre-merge-detection v1

initial situation:

- most e2e tests take 10sec - 2mins to run
- 5 times re-run has “only” 88% chance of detection



# pre-merge-detection v1

problems discovered with v1 approach:

- shotgun approach causes too many tests to be run
  - capping amount of tests re-run required to avoid timeouts
  - CI users complain about failing tests they didn't touch
- re-run lane takes ~1h on average



sources: [KubeVirt e2e test runtimes](#), [check-tests-for-flakes job history](#), [automation/repeated test.sh](#)

The only way to find out the flakiness of a test is to run it as often as you can!

# CANNIER

# CANNIER

**CANNIER** is an approach  
for **reducing the time cost** of rerunning-based detection techniques  
by **combining them with machine learning models**

source: <https://www.gregorykapfhammer.com/research/papers/parry2023/>



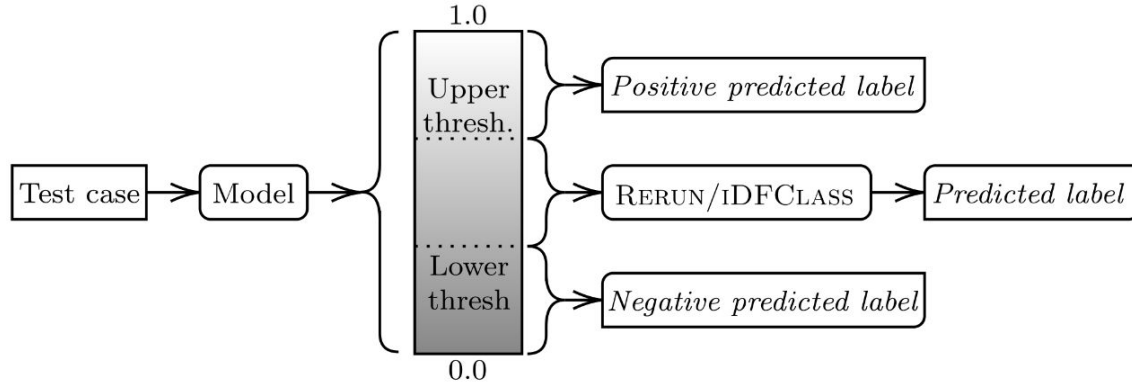
# CANNIER

“... we found that CANNIER was able to reduce the time cost (and therefore monetary cost) *[of re-running tests]* by an **average of 88% ...**”

source: <https://www.gregorykapfhammer.com/research/papers/parry2023/>

# CANNIER

single model



source: <https://www.gregorykapfhammer.com/research/papers/parry2023/>

# CANNIER

feature set (complete)

| #  | Feature               | Description                                                                       |
|----|-----------------------|-----------------------------------------------------------------------------------|
| 1  | Read Count            | Number of times the filesystem had to perform input [9].                          |
| 2  | Write Count           | Number of times the filesystem had to perform output [9].                         |
| 3  | Run Time              | Elapsed wall-clock time of the whole test case execution.                         |
| 4  | Wait Time             | Elapsed wall-clock time spent waiting for input/output operations to complete.    |
| 5  | Context Switches      | Number of voluntary context switches.                                             |
| 6  | Covered Lines         | Number of lines covered.                                                          |
| 7  | Source Covered Lines  | Number of lines covered that are not part of test cases.                          |
| 8  | Covered Changes       | Total number of times each covered line has been modified in the last 75 commits. |
| 9  | Max. Threads          | Peak number of concurrently running threads.                                      |
| 10 | Max. Children         | Peak number of concurrently running child processes.                              |
| 11 | Max. Memory           | Peak memory usage.                                                                |
| 12 | AST Depth             | Maximum depth of nested program statements in the test case code.                 |
| 13 | Assertions            | Number of assertion statements in the test case code.                             |
| 14 | External Modules      | Number of non-standard modules (i.e., libraries) used by the test case.           |
| 15 | Halstead Volume       | A measure of the size of an algorithm's implementation [21, 57, 59].              |
| 16 | Cyclomatic Complexity | Number of branches in the test case code [39, 57, 59].                            |
| 17 | Test Lines of Code    | Number of lines in the test case code [57, 59].                                   |
| 18 | Maintainability       | A measure of how easy the test case code is to support and modify [19, 71].       |

source: <https://www.gregorykapfhammer.com/research/papers/parry2023/>

# CANNIER

## feature set (runtime)

|                        |                                                                                   |
|------------------------|-----------------------------------------------------------------------------------|
| 1 Read Count           | Number of times the filesystem had to perform input [9].                          |
| 2 Write Count          | Number of times the filesystem had to perform output [9].                         |
| 3 Run Time             | Elapsed wall-clock time of the whole test case execution.                         |
| 4 Wait Time            | Elapsed wall-clock time spent waiting for input/output operations to complete.    |
| 5 Context Switches     | Number of voluntary context switches.                                             |
| 6 Covered Lines        | Number of lines covered.                                                          |
| 7 Source Covered Lines | Number of lines covered that are not part of test cases.                          |
| 8 Covered Changes      | Total number of times each covered line has been modified in the last 75 commits. |
| 9 Max. Threads         | Peak number of concurrently running threads.                                      |
| 10 Max. Children       | Peak number of concurrently running child processes.                              |
| 11 Max. Memory         | Peak memory usage.                                                                |

source: <https://www.gregorykapfhammer.com/research/papers/parry2023/>

# CANNIER

feature set (static analysis)

|    |                       |                                                                             |
|----|-----------------------|-----------------------------------------------------------------------------|
| 12 | AST Depth             | Maximum depth of nested program statements in the test case code.           |
| 13 | Assertions            | Number of assertion statements in the test case code.                       |
| 14 | External Modules      | Number of non-standard modules (i.e., libraries) used by the test case.     |
| 15 | Halstead Volume       | A measure of the size of an algorithm's implementation [21, 57, 59].        |
| 16 | Cyclomatic Complexity | Number of branches in the test case code [39, 57, 59].                      |
| 17 | Test Lines of Code    | Number of lines in the test case code [57, 59].                             |
| 18 | Maintainability       | A measure of how easy the test case code is to support and modify [19, 71]. |

source: <https://www.gregorykapfhammer.com/research/papers/parry2023/>

pre-merge-detection v2

# pre-merge-detection v2.0

## questions

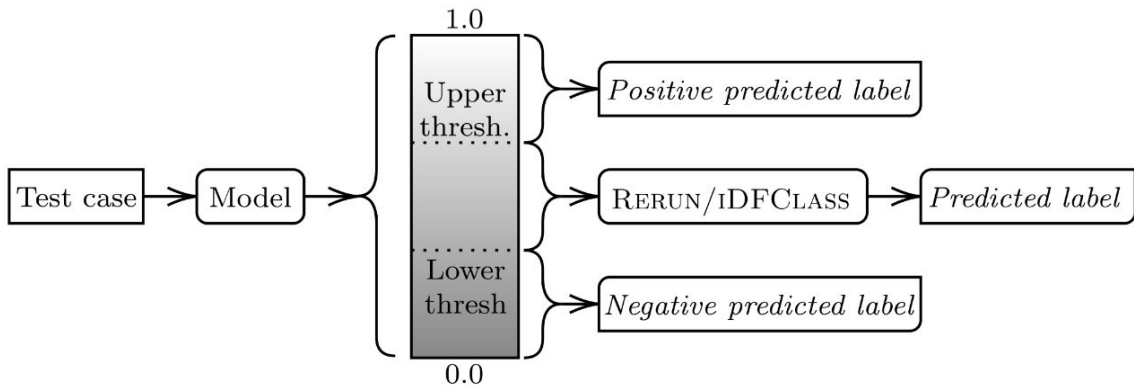
- CANNIER
  - implemented in Python
  - KubeVirt uses Go
- Runtime Data
  - where to store
  - when to capture
- data science
  - Python has well known frameworks
  - Go state unsure

# pre-merge-detection v2.0

## algorithm

### steps:

- gather the set of changed tests
- per each element
  - *run once to get the runtime data*
  - extract the static analysis data
  - fetch the prediction using entire feature vector
  - add to the set of re-run tests if in iDFClass
- re-run the reduced set of tests



source: <https://www.gregorykapfhammer.com/research/papers/parry2023/>

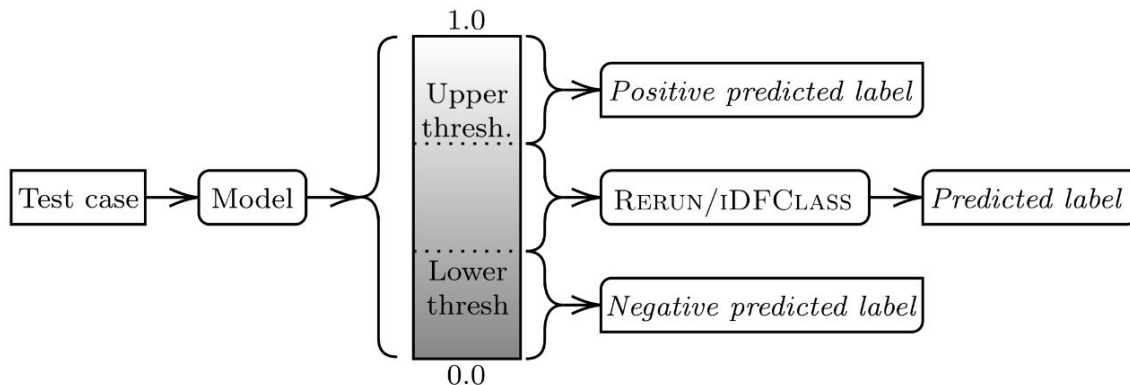


# pre-merge-detection v2.0

## implementation

parts:

- code (Go)
  - test set extraction
  - feature extraction
  - model prediction
  - model generation
- test lane (Bash)
- model deployment (YAML)



source: <https://www.gregorykapfhammer.com/research/papers/parry2023/>

# pre-merge-detection v2.1

improvements

parts:

- model
  - generation
  - deployment
- test lane
- code
  - test set extraction
  - feature extraction
  - model prediction



parts:

- model
  - ...
  - **automatic updates**
- ~~test lane~~ -> **pro external-plugin**
  - runs on presubmits
  - runs on postcommits (probably with a larger test set)
  - adds helpful feedback
- code
  - test set extraction
  - feature extraction
  - model prediction

# pre-merge-detection v2.1

prowl external-plugin

- runs on presubmits
- adds helpful feedback
- gives advice according to the feature set what can be improved

# pre-merge-detection v2.2

## improvements

- components of feature vector provide insightful advice to the contributor
  - i.e. high cyclomatic complexity advises to reduction etc.
  - therefore it's valuable to attach the analysis data to the PR
- possibly increase number of reruns (>5)
  - reduced runtime overall leaves time for more reruns
  - feature vector contains runtimes, thus we can estimate the total re-run time better and optimize for it, i.e. group tests by runtime classes

source: <https://www.gregorykapfhammer.com/research/papers/parry2023/>

pre-merge-detection v1.5 - the present

# pre-merge-detection v1.5 (the present)

approach:

- **gather the exact set of changed tests**
- run these 5 times in random order

# CANNIER

some final remarks:

- CANNIER approach is basically language agnostic
- our implementation is highly dependent on Ginkgo testing framework for Go
- adaptation to default go tests or to other frameworks should be an implementation detail

source: <https://www.gregorykapfhammer.com/research/papers/parry2023/>

# Links

- KubeVirt resources

- Initial pull request (draft): <https://github.com/kubevirt/project-infra/pull/3930>
- Presentation Squash The Flakes @ FOSDEM '24:  
<https://archive.fosdem.org/2024/schedule/event/fosdem-2024-1805-squash-the-flakes-how-to-minimize-the-impact-of-flaky-tests/>

- CANNIER

- paper: <https://www.gregorykapfhammer.com/research/papers/parry2023/>
- implementation: <https://github.com/flake-it/cannier-framework/>



# Q&A

Any questions?

Any suggestions for improvement?

Who else is trying to tackle this problem?

What have you done to solve this?

download slides:



[slides.pdf](#)

# Thank you for attending!

## Further questions?

Feel free to send questions and comments to:

[dhiller@redhat.com](mailto:dhiller@redhat.com)

 [kubernetes.slack.com/  
@dhiller](https://kubernetes.slack.com/@dhiller)

 [@dhiller@fosstodon.org](https://fosstodon.org/@dhiller)

[dhiller.dev](https://dhiller.dev)

contact me:



[kubevirt.io](https://kubevirt.io)

## Virtualization for Kubernetes

### KubeVirt welcomes all kinds of contributions!

- **Weekly community meeting every Wed 3PM CET**
- Links:
  - [KubeVirt website](https://kubevirt.io)
  - [KubeVirt user guide](#)
  - [KubeVirt Contribution Guide](#)
  - [GitHub](#)
  - Kubernetes Slack channels
    - [#virtualization](#)
    - [#kubevirt-dev](#)

